

TECHNICAL DESIGN DOCUMENT

Clinical RAG Prototype

Design & Decision Rationale

A privacy-first, multi-agent retrieval system on Microsoft Azure that answers clinical questions grounded strictly in de-identified patient records, enforces per-patient access control, and treats privacy, cost, and latency as one coherent design decision.

The guiding idea

Sending the model only the minimal, de-identified context it needs is simultaneously the privacy control, the cost optimization, and the latency optimization. Almost every decision in this document follows from that.

How to read this

Each section states a decision and the reasoning behind it. The next page shows the complete architecture at a glance; the sections that follow zoom into each layer. The runnable source lives in the linked repository.

Prepared for technical review | June 2026

Complete System Architecture

The system reads as four stacked layers - interface, orchestration, tools, and data - with three concerns that cut across all of them. The interface drives the two pipelines; the multi-agent orchestration sits on the tools and services; everything resolves against a single Azure AI Search index. Security, evaluation, and cost-and-latency are not a layer but properties enforced throughout.

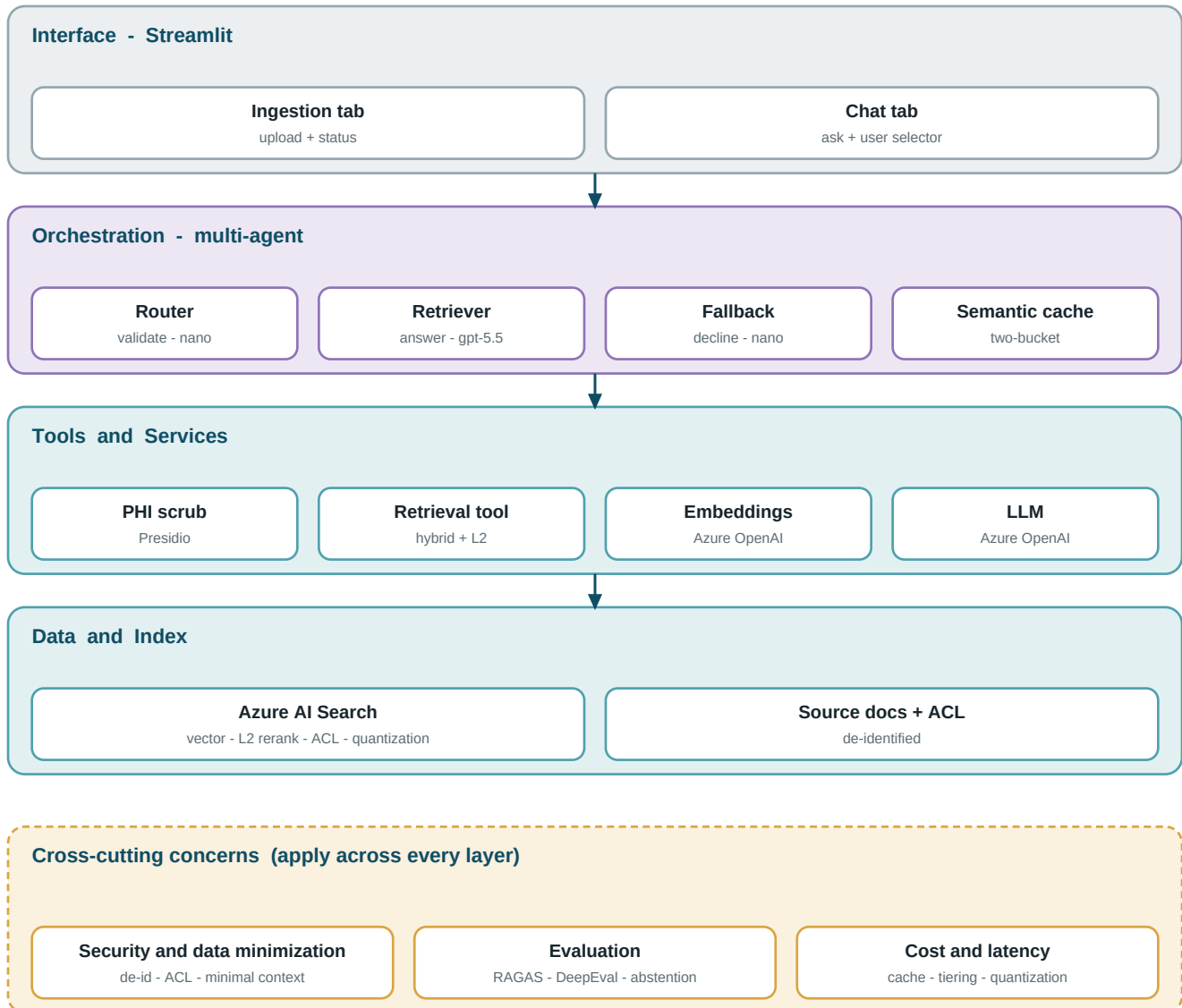


Figure 1. Complete system architecture - four layers plus cross-cutting concerns.

1 Objective and review lens

The goal is a clinical chatbot that answers questions grounded strictly in de-identified patient records, never inventing clinical facts, and never exposing a record to a user who is not authorized to see it. The design was shaped by six concerns a reviewer would weigh: RAG design, evaluation rigor, seriousness about data minimization, retrieval quality, cost, and latency. The system is built as a minimal but production-shaped prototype; where a full production build would differ, the production path is documented rather than half-implemented.

2 The two pipelines

The system splits into two pipelines with deliberately different cost shapes. The **ingestion** pipeline runs once when data changes - slow, cheap, fully deterministic, with no language model in the loop. The **retrieval** pipeline runs on every request, so every optimization is concentrated there.

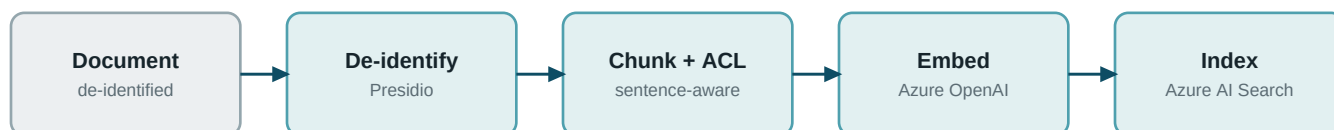


Figure 2. Ingestion pipeline - deterministic, runs once.

On the query side, a deterministic privacy step precedes a three-agent orchestration: the query is scrubbed of identifying detail before it reaches any model, cache, or index.

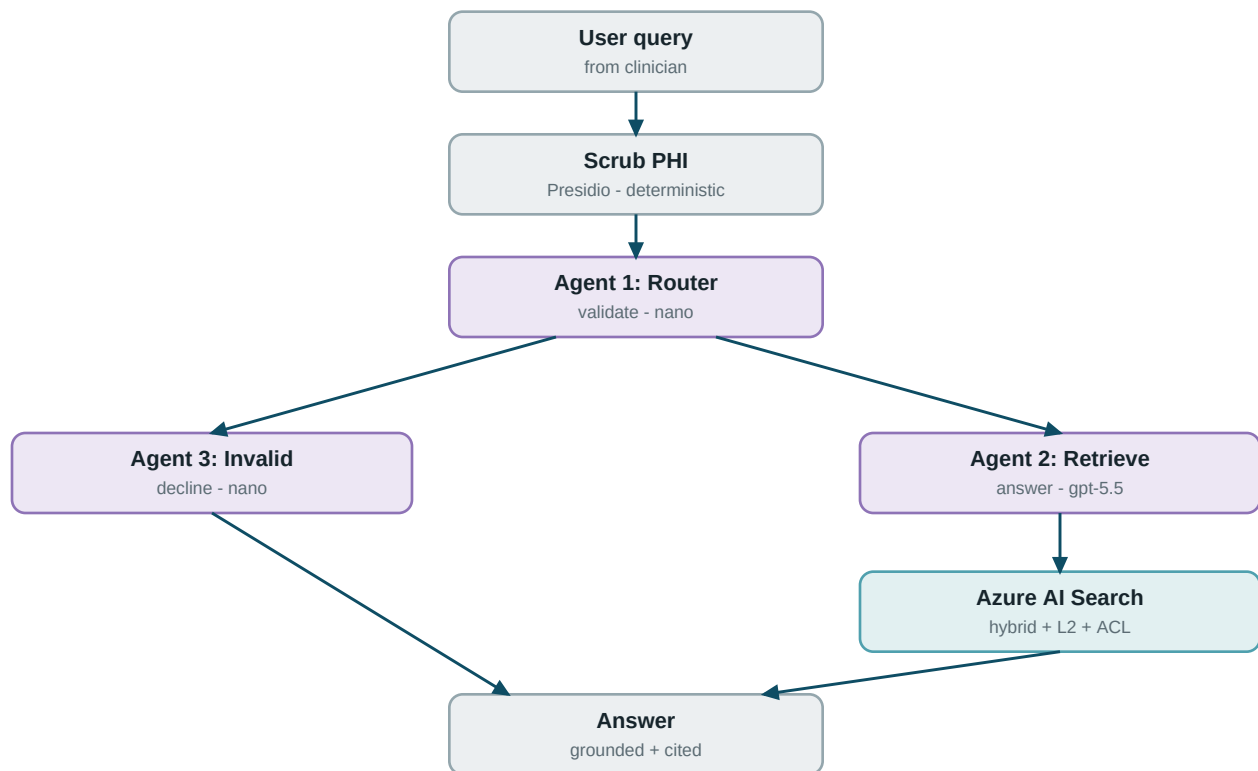


Figure 3. Multi-agent retrieval pipeline.

3 De-identification and data minimization

De-identification is performed by a dedicated named-entity recognition tool (Presidio, or Azure Health Data Services in production), not by a language model. On clinical text, specialized de-identification reaches roughly 91 to 96 percent detection F1, whereas a general LLM trails near 79 percent - and is slower and costlier for the task. The approach follows the Safe Harbor standard and uses consistent surrogate tokens, so the same patient always maps to the same pseudonym without the system ever learning the real identity.

Why minimization is the throughline

Pseudonymize at ingestion so the index holds only de-identified text; scrub the incoming query so a typed name never reaches the model; send only the top reranked chunks; keep raw data out of logs. The same restraint also cuts tokens, cost, and latency.

4 Chunking strategy

Three strategies were considered: fixed-size (simple, but can cut a sentence in half), semantic or meaning-based (most coherent, but it costs extra embedding calls and is overkill for short notes), and sentence-aware. We chose **sentence-aware** chunking - whole sentences packed to a token budget, never cut mid-sentence - which keeps each chunk coherent at almost no cost, and makes overlap optional. The access list and patient identifier are attached to **every chunk** here, because retrieval filters at the chunk level; if one chunk lost its ACL it could leak.

5 Embeddings and dimensionality

Medical accuracy argues for a strong embedding model, while cost and latency argue for smaller vectors. Matryoshka representation learning reconciles them: a high-quality model is embedded once at full size, then truncated. Quality degrades gently - about 95 percent retained at 1024 dimensions, 85 percent at 256 - so the chosen dimension is validated on the evaluation set rather than guessed.

6 Retrieval and reranking

Retrieval is hybrid - keyword and vector fused by reciprocal rank fusion - followed by the Azure AI Search semantic reranker as a second-stage ranker. The built-in reranker was chosen over a separate reranking API for these reasons:

Reason	Benefit
Compliance	Retrieval and reranking stay inside the Azure boundary - no extra vendor or data-sharing agreement.
Latency	No additional network hop to an external reranker service.
Abstention signal	The reranker score is a built-in confidence gate; a low score triggers a safe 'not enough information' response.
Traceability	Semantic captions highlight which part of the source matched - the citation a clinician needs.
Cost	A monthly free tier covers the prototype; usage-billed and toggleable thereafter.

7 Cutting cost without losing quality

Because clinical accuracy cannot be traded away, the headline cost lever is vector quantization with rescoring: stored vectors are compressed (scalar quantization cuts the vector index roughly fourfold), but quality is recovered at query time by oversampling on the compressed vectors and rescoring against the originals - about a 75 percent storage reduction with search quality preserved. The other levers are quality-neutral.

Lever	Effect	Type
Quantization + rescoring	~75% storage cut, quality preserved	Cost
Matryoshka dimensions	Smaller vectors, validated on eval	Cost / latency
Model tiering	Expensive model only on valid answers	Cost
Minimal context	Privacy, cost, and latency at once	All three
Prompt caching	~90% off the cached system prompt	Cost
Batch API (offline eval)	~50% cheaper for non-real-time work	Cost
Data Zone Standard	In-boundary processing + steady latency	Compliance / latency

8 Multi-agent orchestration

A cheap, fast **router** validates the query and classifies it as general or patient-specific. Valid queries go to a **retriever** on the capable model, which uses tool-calling to invoke the search tool and compose a grounded, cited answer; invalid ones go to a cheap **fallback**. The expensive model runs only on the step that needs it, gated by a cheap router.

The security boundary in tool-calling

The retriever may choose the search text, but the user's identity is injected server-side - never by the model. The model can decide what to search for, but not whose records it may see. When nothing confident comes back, the agent abstains rather than guessing.

9 Access control

Records are patient-specific, so access is modeled per patient: each chunk carries a list of the user IDs allowed to read it (the patient plus their care-team examiners). The security trim runs **inside the retrieval query**, so an unauthorized chunk cannot reach the reranker, the prompt, or the cache - access control is enforced before any content moves. The prototype implements a lightweight filter; production maps this onto Entra ID document-level access control.

10 Semantic caching

Caching is the largest cost-and-latency lever, but it must be designed for medical data. The key realization: a semantic cache is keyed on the **meaning** of the question, shared across users, so it scales with the number of distinct questions, not users - ten thousand users asking the same general question collapse to one entry. Per-user caching would be the trap.

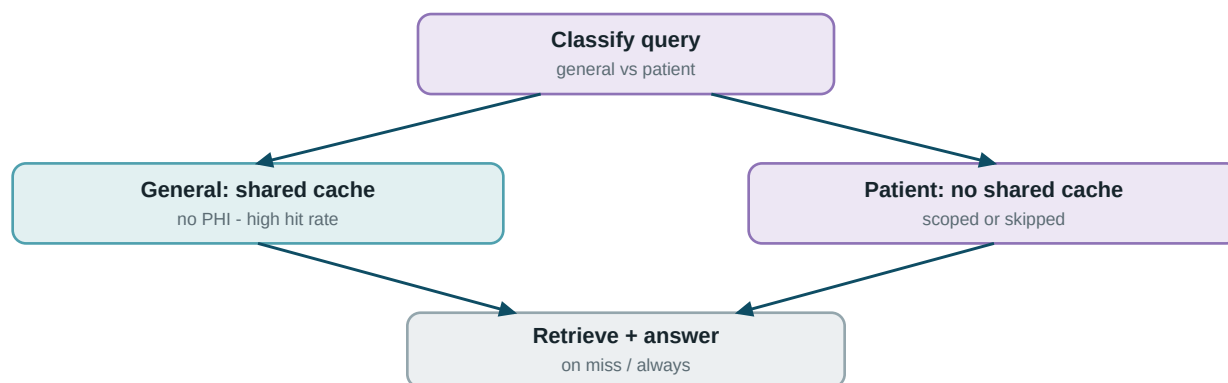


Figure 4. Two-bucket cache (general answers can short-circuit on a hit).

So the cache is split: general-knowledge answers - no personal data - are shared-cached with a deliberately tight similarity threshold (a false hit in medicine is a safety bug, not a quality dip). Patient-specific answers are never shared-cached; they always go to fresh, ACL-filtered retrieval.

11 Evaluation strategy

Two complementary frameworks are used, each for its strength. **RAGAS** judges retrieval (context precision and recall): was the right context surfaced? **DeepEval** judges the agent's answer (faithfulness, answer relevancy, and a custom clinical-correctness criterion). Refusal cases get a separate deterministic check, because when the correct behavior is to decline, faithfulness does not apply.

The headline result

Among the refusal cases are ACL-negative ones - an answerable question asked by an unauthorized user, which must be refused. That single pass/fail is the result that proves the access control works rather than merely claiming it.

12 Golden question set

A test set written by whoever knows the data tends to mirror its wording, which flatters retrieval. Questions are therefore generated from the source documents with an LLM, using styling that forces paraphrase and evolutions that add realistic complexity - never generated from the retrieval system itself, which would grade the system against its own output. Generation cannot invent 'ask about a patient you are not authorized for' or 'ask for a fact that is absent,' so those highest-signal cases stay hand-authored as a curated safety core. Every generated item is a candidate a clinician reviews before it becomes ground truth.

13 Production and scaling

The prototype is minimal but designed to scale. The serving tier is stateless, so it runs multiple replicas and scales to near-zero when idle. The documented production path includes:

- Azure Data Zone Standard - keeps processing in the regional boundary for compliance, with steady latency.
- Provisioned throughput for predictable tail latency and lower cost at sustained volume; pay-as-you-go for bursty use.
- Azure Cache for Redis as the production semantic-cache backend.
- Entra ID document-level access control as the production form of the ACL.
- Response streaming to improve perceived latency at no extra cost.

14 Alignment to review criteria

Criterion	How it is addressed
RAG design	Hybrid + L2 semantic retrieval, sentence-aware chunking, confidence-gated generation, clean two-pipeline split.
Evaluation rigor	RAGAS retrieval metrics, DeepEval answer metrics, deterministic abstention including ACL-negative cases.
Data minimization	Deterministic de-id of records and queries, minimal context, patient-scoped cache, in-boundary services.

Retrieval quality	Semantic L2 reranking, quantization with rescoring, reranker-score abstention.
Cost	Semantic cache, model tiering, quantization, prompt caching, batch eval, free tiers.
Latency	Built-in reranker, cache short-circuit, minimal context, streaming, in-region co-location.

15 What was deliberately deferred

Restraint is part of the design. Multi-turn conversation, a full identity and role-management stack, and heavier agentic orchestration were consciously left out: they add latency, cost, and complexity that work against the very criteria being assessed. Each is noted as a clear next step rather than a half-built feature.

The complete, runnable source - ingestion, the multi-agent chat, the evaluation suite, and the synthetic dataset - is available in the linked repository.